1ICVS415

AY2025-26

Web Technology

Quality Laboratory Manual

ExperimentNo. 09

Program to manage session in PHP

Course Instructor–

Mrs.Amruta R.Patil

ASSISTANT PROFESSOR

ExperimentNo.09

Title of Experiment: Program to manage session in PHP

Aim of Experiment: To design and implement a secure session management system in PHP for maintaining user state across multiple web pages.

System Requirements –

- Operating System: Windows 10 / Linux / macOS
- RAM: Minimum 4 GB
- Processor: Intel i3 or above

Software/s Needed for Experiment-

- Web Browser (Chrome / Firefox)
- Code Editor (VS Code / Sublime Text)
- XAMPP / WAMP / LAMP Server
- PHP 7.x or above

Experiment Objectives:

- To understand server-side session management in PHP.
- To implement secure session handling for user authentication.
- To manage session lifecycle (creation, usage, destruction).
- To prevent unauthorized access using session validation.
- To understand security aspects like session hijacking and session fixation.

Experiment Outcomes:

- Design and implement a session-based authentication system.
- Maintain user state across multiple web pages.
- Securely handle login and logout operations.
- Apply session validation to restrict unauthorized access.
- Understand real-world applications of session management in web systems.

Theory:

Session management is a fundamental concept in web development used to maintain user-specific data across multiple HTTP requests. Since HTTP is a stateless protocol, sessions help in preserving user state.In PHP, sessions are implemented using unique session IDs stored on the client side (usually via cookies) and session data stored on the server.

Key Concepts:

- **Session Initialization:** `session_start()` creates or resumes a session
- **Session Variables:** Stored using `$_SESSION` superglobal
- **Session Termination:** `session_destroy()` clears session data
- **Session Security:** Includes regeneration of session ID and validation

Program

1. login.php

```
<?php
session_start();

// Prevent session fixation
session_regenerate_id(true);

if(isset($_POST['login'])) {
    $username = $_POST['username'];

    // Simple validation (for demo)
    if(!empty($username)) {
        $_SESSION['user'] = $username;
        header("Location: dashboard.php");
        exit();
    }
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Login</title>
</head>
<body>

<h2>Login System</h2>

<form method="post">
Username: <input type="text" name="username" required><br><br>
<input type="submit" name="login" value="Login">
</form>

</body>
</html>
```

2. dashboard.php

```
<?php
session_start();

// Session validation
if(!isset($_SESSION['user'])) {
    header("Location: login.php");
    exit();
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Dashboard</title>
</head>
<body>

<h2>Welcome, <?php echo $_SESSION['user']; ?></h2>
```

```
<p>This is a secure dashboard page.</p>

<a href="logout.php">Logout</a>

</body>
</html>
```

3. logout.php

```
<?php
session_start();

// Unset all session variables
session_unset();

// Destroy session
session_destroy();

header("Location: login.php");
exit();
?>
```

Task for Students: Students will design and implement a secure session-based authentication system using PHP.

Task Requirements:

- Develop Login System
- Implement Session Handling
- Create Protected Pages
- Implement Logout Functionality
- Enhance Security

Submission Requirement:

- PHP Source Code (All Files)
- Screenshot of Login Page
- Screenshot of Dashboard (Session Working)
- Screenshot of Logout Process
- Brief Report (Explanation + Flow)

Observations:

The experiment demonstrated the use of PHP sessions for maintaining user authentication state. Session variables were used to store user data securely, and access control was implemented to restrict unauthorized users. Security enhancements such as session regeneration improved system reliability.

Conclusion:

Session management is essential for building secure web applications. This experiment provided practical exposure to authentication mechanisms and highlighted the importance of session security in real-world systems.

Expected Oral Questions:

1. What is session management?
2. Why is HTTP stateless?

3. What is `session_regenerate_id()`?
4. How do sessions improve security?
5. What is session hijacking?
6. Difference between cookies and sessions?
7. How to secure session data?

FAQs in Interview:

1. How does PHP session work internally?
2. What is session fixation attack?
3. How can you prevent session hijacking?
4. What are best practices for session management?
5. Where is session data stored in PHP?
6. Can sessions work without cookies?
7. What is the role of session ID?
8. How do large applications manage sessions?